LEEDS TRINITY UNIVERSITY

Computer Science

Trinity Digital Tech Programme

Activities & Projects Brief

Artificial Intelligence · Cyber Security · Game Development

A scaffolded, hands-on programme taking students from first principles to team-built capstone projects across three computing disciplines.

Contents

Programme overview

This programme introduces students to three of the most exciting areas of modern computing, Artificial Intelligence, Cyber Security and Game Development, through a carefully sequenced set of hands-on activities. Each theme runs as its own learning journey of ten short, practical activities, followed by a choice of team capstone projects that put the new skills to work.

The activities are deliberately small and self-contained so that every student, regardless of prior experience, can complete each one and feel a sense of progress. They use free, accessible tools, browser-based games written in HTML, JavaScript and the p5.js library, and Python notebooks (Google Colab and Jupyter), so there is nothing to install and the work is easy to share.

The design follows a scaffolded approach: early activities establish core ideas with plenty of support, and each subsequent activity removes a little of that support while adding one new concept. By the end of a theme, students are combining several skills at once and are ready to take on an open-ended project with their peers.

How scaffolding works across the programme

Scaffolding is a teaching approach in which complex learning is broken into manageable steps, with structured support that is gradually withdrawn as the learner becomes more capable. In this programme that principle shapes the whole sequence.

- **Start concrete and rewarding.** The first activity in every theme is something a student can finish quickly and enjoy, building confidence before any difficulty.
- **One new idea at a time.** Each activity reuses skills from the previous one and introduces a single new concept, so cognitive load stays manageable.
- **From guided to independent.** Early activities are heavily worked through; later ones ask students to combine techniques with less direction.
- **Visible progression of difficulty.** Activities are tagged Beginner, Intermediate or Advanced so the climb is clear to students and staff alike.
- **Apply it for real.** The capstone projects require students to select, combine and extend what they have learned, the point at which the scaffolding comes away and genuine independent skill is demonstrated.
- **Connections across themes.** The themes reinforce one another, for example pathfinding in Game Development echoes AI search, and the Typing Speed Game becomes the basis of an AI analyser, helping students transfer skills between domains.

At a glance

Theme	Journey from first activity to capstone	Capstone outcome
Artificial Intelligence	Number guessing to pre-trained text & vision models to generative AI to multimodal captioning	Chatbot, typing analyser or image-classifier web app

Theme	Journey from first activity to capstone	Capstone outcome
Cyber Security	Password safety to classical ciphers to web attacks (XSS, SQLi) to modern crypto & 2FA to social engineering	Vulnerable-form challenge, encryption toolbox or secure chat
Game Development	Clicker to animation & collisions to procedural obstacles to progression & persistence to real-time audio control	Level-based runner, maze explorer/solver or multiplayer score battle

Theme 1 - Artificial Intelligence

The AI theme demystifies modern machine learning by letting students use powerful pre-trained models from day one. Rather than starting with heavy theory, learners call working models to analyse text, recognise images and even generate art, then gradually uncover how those models actually work.

The sequence moves deliberately between the two great strands of AI, language and vision, before uniting them at the end, so students leave with an intuitive grasp of what today's AI systems can and cannot do.

How the activities scaffold learning

The journey opens with a pure-logic Number Guesser to settle everyone into Python, then immediately shows the payoff of modern AI with a one-line Sentiment Analyser. From there, classification ideas deepen through the Emoji Predictor and Image Classifier, broadening from text to vision.

The midpoint adds richer interaction (the live Pose Counter) and generation (the Text Generator), before the Teachable Machine activity pulls back the curtain to reveal how classification and training actually work, reframing everything that came before. The theme then peaks with two advanced generative activities, the AI Art Generator and the multimodal Image Caption Generator, which combine the language and vision skills students have accumulated and lead naturally into the capstone projects.

The activities

1. Number Guesser BEGINNER

Students build a text-based game in which the computer picks a random number and the player guesses, receiving 'higher' or 'lower' feedback until they win. It is the gentlest possible entry point: no libraries, no maths, just the core programming ideas every later activity depends on. Learners meet variables, user input, conditionals and loops in a context that gives instant, satisfying feedback.

Key skills: Random number generation, input validation, conditional logic, loops and user interaction.

2. Sentiment Analysis BEGINNER

Learners use a pre-trained model from the HuggingFace transformers library to judge whether a sentence they type is positive or negative. This is the moment students realise modern AI is something they can call in a few lines of code. It introduces natural language processing and the idea of a model returning a prediction, without requiring any training of their own.

Key skills: Natural language processing, sentiment classification, using HuggingFace transformers, Python functions and inputs.

Builds on: The Python fundamentals from the Number Guesser.

3. Emoji Predictor BEGINNER

A playful extension of sentiment work: the model reads the user's text and predicts the most fitting emoji, alongside a confidence score. Students see that classification is not limited to two labels and begin to read model confidence critically, asking how sure the AI really is.

Key skills: NLP fundamentals, text classification, pre-trained transformer models, interpreting confidence scores.

Builds on: The sentiment classification activity.

4. Image Classifier INTERMEDIATE

Students move from text to vision, feeding an image into a pre-trained HuggingFace vision model and reading back its top predictions. They handle image input in the notebook environment and meet the idea of tensors. This is the conceptual backbone of the advanced capstone Image Classifier Web App.

Key skills: Image classification, using HuggingFace vision models, handling image input, tensor operations.

Builds on: The text-classification activities, transferring the 'model returns a prediction' pattern to images.

5. Pose Counter with Webcam INTERMEDIATE

Using MediaPipe and OpenCV, learners track body posture through a webcam and count exercise repetitions such as squats by measuring joint angles. It is a hands-on, physical demonstration of real-time computer vision that consistently captures attention, while quietly teaching geometry, video capture and counting logic.

Key skills: Pose estimation with MediaPipe, webcam and video capture, joint-angle calculation, counting logic.

Builds on: Image handling from the Image Classifier, now applied to live video.

6. AI Text Generator INTERMEDIATE

Students prompt a pre-trained language model to continue or generate text, experimenting with how the wording of a prompt changes the output. This introduces natural language generation and the foundational skill of prompt design that underpins today's generative AI tools.

Key skills: Natural language generation, text-generation models, prompt design, language-model basics.

Builds on: NLP understanding from the sentiment and emoji activities.

7. Teachable Machine (Simulated) INTERMEDIATE

A deliberately simplified classifier that demonstrates how 'teaching' a machine works: students supply labelled examples and watch the logic classify new input. By building the idea by hand, learners understand supervised learning and training data before relying on automated tools, which demystifies what earlier pre-trained models were doing internally.

Key skills: Classification logic, simulating training with examples, manual data labelling, the concept of supervised learning.

Builds on: Every prior classification activity; it reveals the mechanism behind them.

8. AI Rock-Paper-Scissors BEGINNER

A lightweight, confidence-building game in which the computer makes a random choice and the program decides the winner. Placed mid-sequence as a consolidation activity, it lets students cement game-logic and input-handling skills that feed directly into the Game Development theme.

Key skills: Random selection, conditional logic, game-rule implementation, comparing player versus computer choices.

Builds on: Logic foundations from the Number Guesser.

9. AI Art Generator (Text-to-Image) ADVANCED

Learners use HuggingFace Diffusers to turn written prompts into original images, experiencing generative AI at its most striking. The activity deepens prompt-engineering skill and opens a discussion about creativity, bias and the responsible use of generative models.

Key skills: Generative AI for images, prompt engineering, text-to-image translation, using the diffusers and transformers libraries.

Builds on: Prompt design from the Text Generator.

10. Image Caption Generator ADVANCED

The capstone activity for the theme: a vision-language model that looks at an image and writes a descriptive sentence about it. It unites the vision and language strands students have built throughout the sequence, showing how modern multimodal AI combines both, and is an ideal springboard into team projects.

Key skills: Vision-language model usage, image understanding, caption generation, image-to-text tasks with transformers.

Builds on: Both the Image Classifier (vision) and the Text Generator (language).

Capstone projects

After the activities, students choose one team project that extends a familiar activity into a complete application. The three options map onto the Beginner, Intermediate and Advanced strands of the theme.

Project 1 - Emotion-Based Response Bot BEGINNER

Builds on: the Sentiment Analysis activity

Goal: Create a simple chatbot that detects the emotion in a user's message and replies in a way that matches the mood.

Suggested team roles: NLP / sentiment detection; chatbot interface design.

Key steps: Use the sentiment example as a base, build a chat interface, and add logic that responds positively to negative input.

Project 2 - AI Typing Speed Analyser INTERMEDIATE

Builds on: the Typing Speed Game plus basic ML logic

Goal: Build an app that measures typing speed and uses simple AI to detect improvement over time and give feedback.

Suggested team roles: Typing-game mechanic; statistics tracker; feedback logic.

Key steps: Capture words-per-minute, errors and time, analyse for improvement, then present a visual dashboard.

Project 3 - AI Image Classifier Web App ADVANCED

Builds on: the Image Classifier and broader AI concepts

Goal: Create an image classifier (for example plants vs animals vs objects) that lets users upload their own images.

Suggested team roles: Front-end image upload; back-end model classification; results UI with a confidence graph.

Key steps: Use MobileNet or Teachable Machine, build an upload form, and display predictions with confidence scores.

Theme 2 - Cyber Security

The Cyber Security theme builds 'security thinking', the habit of asking how a system could be misused and how to defend it. Every activity pairs an attack with its defence, so students never learn to break something without also learning to protect it.

The work is delivered through safe, self-contained simulations in the browser and in Python notebooks, allowing learners to witness real vulnerabilities, such as SQL injection and cross-site scripting, in a controlled, ethical setting.

How the activities scaffold learning

The theme begins with the universally familiar topic of passwords, establishing input-validation habits, then introduces encryption gently with the Caesar Cipher, including why it is weak. That weakness motivates the central run of web-attack activities, HTML/JavaScript injection, XSS and SQL injection, each building the same core lesson that all input is untrusted.

Students then strengthen their toolkit with hashing and modern Fernet encryption, replacing the weak cipher from earlier, and add layered defence with the Two-Factor Authentication simulator. Finally the theme widens from technical attacks to the human factor through the Phishing and Social Engineering activities, reinforcing that security is about people as well as code. Each strand feeds a matching capstone.

The activities

1. Password Strength Checker & Meter BEGINNER

Students build a tool (in Python and as an interactive web page) that scores how secure a password is against criteria such as length, mixed case, numbers and symbols. Starting with something every learner already has, passwords, makes security personal and immediate, and establishes the input-validation habits the whole theme relies on.

Key skills: Input validation, string handling, basic security practice, awareness of password-strength criteria.

2. Caesar Cipher Encoder / Decoder BEGINNER

Learners encrypt and decrypt messages by shifting characters along the alphabet, implementing one of the oldest known ciphers. It introduces the core idea of encryption in a tangible way and, crucially, shows why simple ciphers are weak, motivating the stronger methods that follow.

Key skills: Basic encryption logic, character shifting, working with strings and ASCII values, understanding the vulnerability of simple ciphers.

Builds on: String-handling skills from the Password Checker.

3. HTML & JavaScript Injection BEGINNER

Working in the browser, students see what happens when user input is dropped onto a page without checks. They watch their own markup and scripts take over a page, a memorable first encounter with the principle that all input is untrusted, which directly sets up the XSS activity.

Key skills: Client-side rendering, the danger of unescaped input, the principle of input sanitisation.

Builds on: Validation thinking from the Password Checker, now in a web context.

4. Cross-Site Scripting (XSS) Simulation INTERMEDIATE

In a safe sandbox, students post a comment containing a script and watch an 'unsafe' page execute it, then compare a 'safe' version that sanitises the input. This makes one of the web's most common vulnerabilities concrete and teaches the defensive coding that prevents it.

Key skills: Understanding cross-site scripting, the risks of innerHTML, the importance of input sanitisation.

Builds on: The HTML/JavaScript Injection activity.

5. SQL Injection Simulation INTERMEDIATE

Learners exploit a deliberately vulnerable login form by entering input such as ' OR '1'='1 to bypass authentication, then fix it with parameterised queries. It demonstrates why separating code from data matters and is the direct foundation for the 'Hack Me If You Can' capstone.

Key skills: How SQL injection works, why it is dangerous, secure-coding practices and parameterised queries.

Builds on: The injection mindset from the XSS activity.

6. Hash Cracker & Hashing Challenge INTERMEDIATE

Students learn how hashing turns data into a fixed fingerprint, then run a simple brute-force demonstration to recover a weak value, strictly for education. This clarifies the difference between hashing and encryption and reinforces why strong, unusual passwords matter.

Key skills: Understanding hashing (MD5, SHA-256), brute-force demonstration, the importance of strong passwords.

Builds on: Password and encryption concepts from activities 1 and 2.

7. Encryption & Decryption Tool (Fernet) INTERMEDIATE

Moving beyond the Caesar cipher, learners use symmetric encryption with the Fernet library to securely encrypt and decrypt messages with a key. They experience properly strong, modern cryptography and prepare for the Encryption Toolbox and Secure Chat capstones.

Key skills: Symmetric encryption basics, secure message handling, working with keys.

Builds on: The Caesar Cipher, replacing a weak scheme with a strong one.

8. Two-Factor Authentication Simulator INTERMEDIATE

Students simulate a 2FA flow: after a password, a one-time code is 'sent' and must be entered before a time limit. It connects the password work to a layered, real-world defence and introduces time-bounded validation.

Key skills: Understanding two-factor authentication, random code generation, timed validation.

Builds on: Password-security foundations from activity 1.

9. Phishing Detection Quiz & Detector BEGINNER

Through realistic examples, learners decide whether a message is legitimate or a phishing attempt and learn the tell-tale signs. The theme widens from technical attacks to the human factor, building the critical scepticism that protects users in practice.

Key skills: Recognising phishing traits, awareness of social engineering, critical evaluation of messages.

Builds on: Security awareness threaded through earlier activities.

10. Social Engineering Scenarios & Game ADVANCED

The capstone activity presents scenarios, pretexting, baiting, urgency, in an interactive game where students choose how to respond and discuss the psychology behind each trick. It consolidates the message that security is as much about people as technology and is ideal for group discussion.

Key skills: Psychological manipulation tactics, defending through scepticism, applied security decision-making.
Builds on: The phishing activity, broadened to the full spectrum of human-factor attacks.

Capstone projects

The capstone projects ask teams to build, attack and defend, putting the attack-and-defence pattern of the activities into practice. The three options align with the Beginner, Intermediate and Advanced strands.

Project 1 - Hack Me If You Can BEGINNER

Builds on: the SQL Injection and XSS activities
Goal: Create a mock login form with known vulnerabilities, let other groups try to break it, then secure it.
Suggested team roles: Form developer; security tester; patch writer.
Key steps: Build a vulnerable form, demonstrate the attacks, then fix them with input sanitisation.

Project 2 - Encryption Toolbox INTERMEDIATE

Builds on: the Caesar Cipher and Fernet encryption activities
Goal: Build a tool that encrypts and decrypts messages using Caesar, ROT13 and Fernet.
Suggested team roles: Python encryption coder; UI builder.
Key steps: Start from the Caesar cipher, add Fernet, and let users toggle between methods.

Project 3 - Secure Chat App Simulation ADVANCED

Builds on: multiple activities including encryption and XSS
Goal: Create a simple encrypted chat between two users, in the browser or a notebook.
Suggested team roles: Message-encryption logic; front-end chat interface; session simulation.
Key steps: Build a message UI, encrypt on send and decrypt on receive, and keep a message history.

Theme 3 - Game Development

The Game Development theme is the most immediately creative of the three, and a powerful vehicle for teaching programming because feedback is instant and visible. Students build a series of complete, playable games, each introducing a new mechanic, before combining them into an original project.

All games run in the browser using HTML, JavaScript and the beginner-friendly p5.js library, so students can play and share their work straight away with nothing to install.

How the activities scaffold learning

The theme starts with the simplest possible interactive loop, a Clicker game, then layers in game state (Memory Match), animation and collisions (Catch the Ball) and continuous, procedurally generated play (Dino Runner). A Maze Solver introduces grids and pathfinding, offering a natural link back to AI.

From the midpoint, students add measurement and polish, the Typing Speed Game (reused later by the AI theme), Sprite Animation and a reusable Scoreboard, before tackling the advanced concepts of progression and persistence (Level Unlocker) and real-time audio control (Sound-Controlled Game). By the end, learners have a full toolkit of mechanics to draw on for their capstone.

The activities

1. Clicker Game BEGINNER

Players click a button to earn points in this minimal first game. It is the simplest possible introduction to event handling and updating the screen in response to a user action, giving every learner an early, visible win in the browser.

Key skills: Event handling, JavaScript logic in the browser, basic UI interaction.

2. Memory Match Game BEGINNER

Students build a card-matching game, tracking which cards are flipped and whether they pair. It introduces game state and manipulating page elements (the DOM), a step up in logic from the single-button clicker.

Key skills: Game logic and state, matching algorithms, DOM interaction.

Builds on: Event handling from the Clicker Game.

3. Catch the Ball BEGINNER

Using the p5.js creative-coding library, learners move a paddle to catch falling objects, with a live score. This is the first taste of animation and the game loop, introducing collision detection in an approachable way.

Key skills: Animation with p5.js, the game loop, collision detection, score tracking.

Builds on: Game state from the Memory Match Game.

4. Dino Runner Clone INTERMEDIATE

Students recreate a simplified version of the well-known Chrome dinosaur game, with a jumping character and obstacles that appear over time. It brings together animation, timing and continuously generated hazards, and is the basis for the beginner Game Dev capstone.

Key skills: Character animation, obstacle generation, timing, continuous gameplay.

Builds on: Animation and collision detection from Catch the Ball.

5. Maze Solver AI INTERMEDIATE

Learners visualise how a simple algorithm finds its way through a maze on a grid, step by step, and can also navigate it themselves. It introduces grids, pathfinding and a first, intuitive look at how 'AI' can search for a solution, bridging neatly back to the AI theme.

Key skills: Grid systems, step-by-step logic, pathfinding and basic AI solving techniques.

Builds on: Coordinate and logic skills built across earlier games.

6. Typing Speed Game INTERMEDIATE

Players type displayed text as quickly and accurately as they can while the game tracks words-per-minute and errors. It introduces timers and capturing detailed input data, and is reused as the basis of the AI Typing Speed Analyser capstone in the AI theme.

Key skills: Timer logic, text-input tracking, performance measurement.

Builds on: UI and input handling from earlier activities.

7. Sprite Animation Demo INTERMEDIATE

Students animate a character from a sprite sheet, displaying frames in sequence to create movement in p5.js. This teaches the frame-based animation that gives games their polish and visual life.

Key skills: Frame-based animation, sprite-sheet control, visual presentation.

Builds on: The p5.js foundations from Catch the Ball and the Dino Runner.

8. Scoreboard Counter INTERMEDIATE

Learners build a reusable scoreboard with a timer, the component that turns a toy into a real game with stakes. It focuses on managing and displaying score and time cleanly and feeds directly into the Multiplayer Scoreboard Battle capstone.

Key skills: Score management, timer logic, clean UI updates.

Builds on: Timer work from the Typing Speed Game.

9. Level Unlocker ADVANCED

Students add progression: as the player's score rises, new levels unlock and difficulty scales, with progress remembered between sessions. It introduces persistence and the design thinking behind a satisfying difficulty curve, and combines with the Dino Runner for the beginner capstone.

Key skills: Progression systems, difficulty scaling, saving state (local storage), game design.

Builds on: Score handling from the Scoreboard Counter.

10. Sound-Controlled Game ADVANCED

In the capstone activity, learners control a character with sound from the microphone, selecting an input device and responding to volume in real time. It is an ambitious, attention-grabbing finale that combines real-time input, audio and everything built earlier.

Key skills: Real-time audio input, device selection, interactive feedback, integrating multiple systems.

Builds on: The full toolkit of animation, scoring and interaction from the preceding games.

Capstone projects

For the capstone, teams combine several mechanics into one polished game. The three options correspond to the Beginner, Intermediate and Advanced strands of the theme.

Project 1 - Dino Runner with Level Unlocker BEGINNER

Builds on: the Dino Runner and Level Unlocker activities

Goal: Make the Dino Runner game get harder as the player's score increases.

Suggested team roles: Game logic; UI/UX and level unlocking.

Key steps: Add a level variable driven by score that increases speed and obstacles.

Project 2 - Maze Explorer vs Solver INTERMEDIATE

Builds on: the Maze Solver activity

Goal: Create two game modes: a player-controlled mode and an automatic AI-solve mode.

Suggested team roles: Maze generation and rendering; player logic; AI auto-solver.

Key steps: Build an interactive maze, add a breadth/depth-first auto-solver, and compare the paths.

Project 3 - Multiplayer Scoreboard Battle ADVANCED

Builds on: the Scoreboard Counter, Catch the Ball and Sound-Controlled activities

Goal: Create a fast-paced clicking or keyboard game with head-to-head score battles.

Suggested team roles: Game logic; scoreboard management; timer logic.

Key steps: Combine scoreboard, timer and player controls, add random targets, and declare a winner when time runs out.

Bringing it together

Across all three themes the design is consistent: thirty short activities, ten per theme, each adding a single idea, leading to nine capstone project options that let students apply their learning in teams. The progression from a first guided activity to an independent, self-directed project is exactly the arc that builds durable confidence and skill.

The themes are also mutually reinforcing. Search and pathfinding appear in both AI and Game Development; the Typing Speed Game built in the Game theme becomes the engine for an AI analyser; and the security mindset of questioning assumptions strengthens students' work everywhere. Taken together, the programme offers a coherent, engaging and genuinely educational introduction to computing that is ready to present and to run.